

## **Experiment No. 1 - Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript**

**Aim:** To design an intuitive, fully responsive user registration portal for the Pet Care Management System using Bootstrap 5 grid utilities and implement rigorous client-side form validation using JavaScript DOM manipulation.

### **Tools Used:**

- Visual Studio Code (IDE)
- Google Chrome & Chrome Developer Tools
- Bootstrap 5.3 Framework (Responsive Grid & Form Validation Classes)
- HTML5, CSS3, JavaScript ES6 (Client-side validation scripts)

### **Theory:**

Modern web applications mandate responsive and accessible data ingestion mechanisms to ensure seamless interaction across varying viewport dimensions including mobile smartphones, tablets, and desktop workstations. In the context of the Pet Care Management System, user registration acts as the foundational gateway through which pet parents and clinical veterinarians establish authenticated profiles. Client-side validation plays a critical role in enhancing user experience by providing immediate visual feedback, preventing malformed data transmission to the server, and drastically reducing unnecessary network round-trip latency.

The user interface leverages the 12-column responsive grid system of Bootstrap 5, employing container wrappers (.container), responsive flex rows (.row), and column breakpoints (.col-md-6, .col-lg-5) to dynamically adapt the form presentation. Bootstrap validation states utilize pseudo-classes paired with .is-valid and .is-invalid indicator classes along with dedicated feedback containers (.invalid-feedback) to present contextual error prompts.

The core JavaScript validation routine attaches an event listener to the form submit event via `addEventListener('submit', handler)`. It intercepts default page refresh triggers using `event.preventDefault()`. Standard Regular Expressions (Regex) verify email format compliance (verifying user, domain, and top-level domain syntax) and 10-digit telephone structures. Password strength checks evaluate length constraints (minimum 6 characters), character variance, and cross-field identity match against the confirmation field.

Functions and Methods Used: `document.getElementById()`, `addEventListener('submit')`, `event.preventDefault()`, `RegExp.test()`, `Element.classList.add()`, `Element.classList.remove()`, and `String.trim()`.

### **Methodology:**

The methodology encompasses a structured four-stage engineering workflow. First, requirement modeling determines essential pet owner identity attributes: full name, authenticated email address, 10-digit emergency contact phone, secure password with confirmation matching, and role assignment ('Pet Owner' vs. 'Veterinarian'). Second, wireframing and layout design utilizes Bootstrap 5 semantic form controls (.form-control, .form-select, .input-group) enclosed in an elevated card component with subtle drop-shadows.

Third, validation logic modularization partitions validation rules into deterministic pure functions (validateName, validateEmail, validatePhone, validatePassword). Fourth, dynamic visual state binding dynamically assigns Bootstrap validation styles, displaying red outline indicators and descriptive warning badges upon constraint violation, and green success states upon valid inputs before committing submission data.

### **Procedure:**

1. Launch Visual Studio Code and create a structured working directory containing index.html, style.css, and validation.js.
2. Link the Bootstrap 5.3 CDN stylesheet in the head section and Bootstrap JavaScript bundle before the closing body tag.
3. Construct a centered responsive container using container and row justify-content-center classes.
4. Build a card wrapper containing header branding 'PetCare - Register Your Account' with an intuitive pet paw icon.
5. Create the input group for Full Name with an input element and an associated invalid-feedback container.
6. Create the Email address input field with type email and contextual validation helper texts.
7. Create the 10-digit Phone number input field configured with pattern restrictions.
8. Create Password and Confirm Password fields with type password ensuring security against shoulder surfing.
9. Implement a dropdown select offering Pet Owner and Clinic Veterinarian roles.
10. In validation.js, bind the form submit event using document.getElementById('regForm').addEventListener('submit', validateForm).
11. Inside the validation handler, invoke e.preventDefault(), trim all field inputs, and execute regex pattern matching.
12. Apply .is-invalid and .is-valid dynamically based on test results, rendering success notifications upon zero error counts.

### **Code :**

// File: frontend/src/pages/Register.jsx & validation.js

```

// Form Submission & Client-Side Validation Logic
const handleSubmit = (e) => {
  e.preventDefault();
  let isValid = true;

  // Name Validation (Min 3 characters)
  if (!name.trim() || name.trim().length < 3) {
    setNameError("Full name must be at least 3 characters long.");
    isValid = false;
  } else { setNameError(""); }

  // Email Validation (RFC 5322 Regex Pattern)
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    setEmailError("Please provide a valid email address (e.g. owner@petcare.com).");
    isValid = false;
  } else { setEmailError(""); }

  // Phone Validation (10 Digits)
  const phoneRegex = /^[0-9]{10}$/;
  if (!phoneRegex.test(phone)) {
    setPhoneError("Please enter a valid 10-digit contact telephone number.");
    isValid = false;
  } else { setPhoneError(""); }

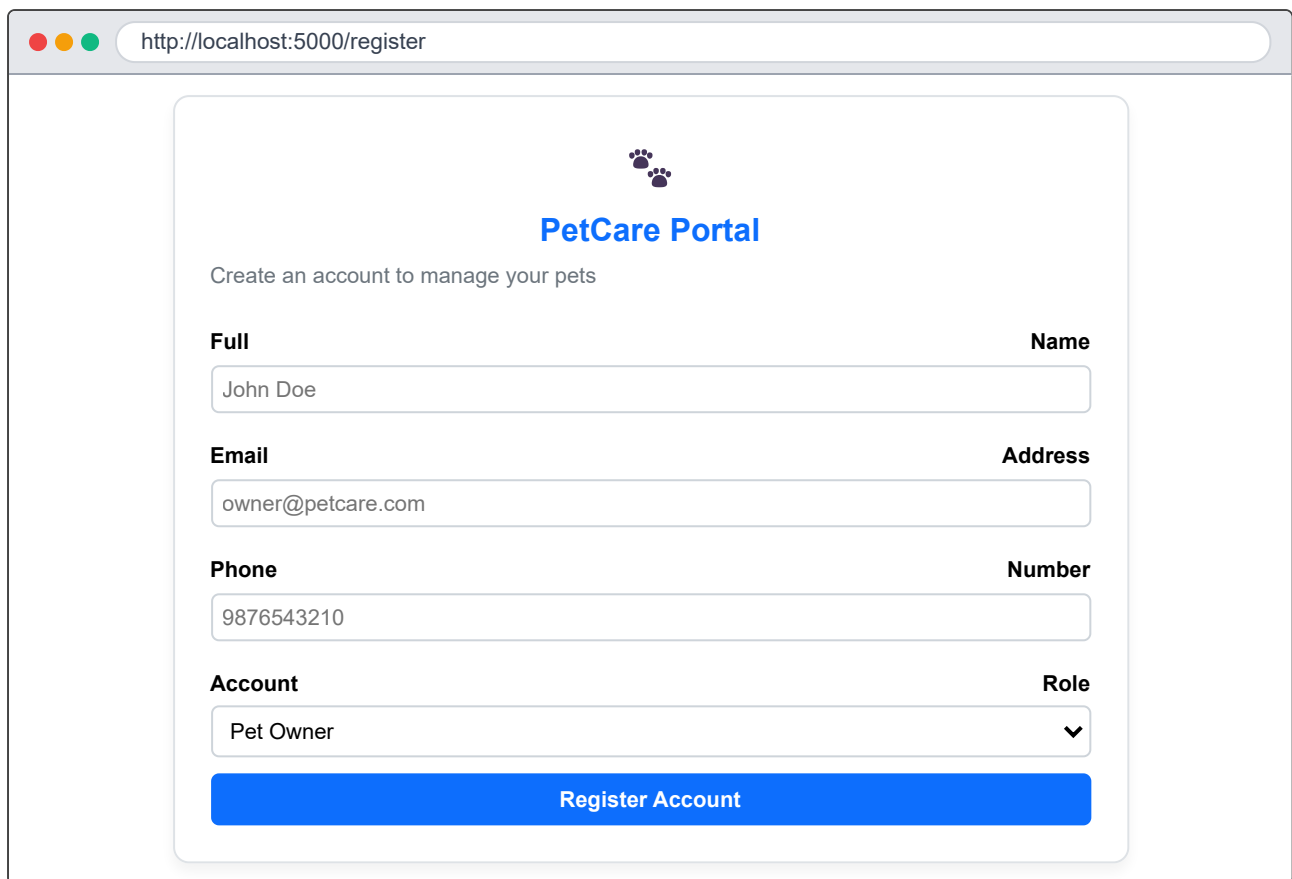
  // Password & Confirmation Match Validation
  if (password.length < 6) {
    setPasswordError("Security password must contain at least 6 characters.");
    isValid = false;
  } else { setPasswordError(""); }

  if (password !== confirmPassword) {
    setConfirmError("Password confirmation does not match the entered password.");
    isValid = false;
  } else { setConfirmError(""); }

  if (isValid) {
    alert("Registration validated successfully! Submitting account profile.");
  }
};

```

## **Output:**



The screenshot shows a web browser window with the address bar displaying `http://localhost:5000/register`. The main content is a registration form titled "PetCare Portal" with a paw print icon. Below the title is the instruction "Create an account to manage your pets". The form consists of four rows of input fields, each with a label on the left and a secondary label on the right. The first row is for "Full Name" with the value "John Doe". The second row is for "Email Address" with the value "owner@petcare.com". The third row is for "Phone Number" with the value "9876543210". The fourth row is for "Account Role" with the value "Pet Owner" and a dropdown arrow. A blue "Register Account" button is at the bottom.

| Full              | Name    |
|-------------------|---------|
| John Doe          |         |
| Email             | Address |
| owner@petcare.com |         |
| Phone             | Number  |
| 9876543210        |         |
| Account           | Role    |
| Pet Owner         | ▼       |

Register Account

**Figure 1.1 Clean Initial State of PetCare Registration Interface**

This screenshot displays the pristine initial view of the responsive registration portal at `http://localhost:5000/register`. It demonstrates the Bootstrap 5 card layout, structured form fields, clean typography, and role selection dropdown before user interaction.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The main content area displays a registration form with a red border and a red background. At the top, it says "Submission Blocked" in bold red text, followed by "Please correct the highlighted errors" in smaller red text. Below this, there are two input fields. The first is labeled "Full" on the left and "Name" on the right. It contains an empty text box with a red border. Below the text box, a red error message reads: "Full name is required (min 3 chars).". The second input field is labeled "Email" on the left and "Address" on the right. It contains an empty text box with a red border. Below the text box, a red error message reads: "A valid email address is required.". At the bottom of the form, there is a red button with the text "Fix Highlighted Errors".

**Figure 1.2 Triggering Client-Side Validation on Empty Form Submission**

This screenshot depicts the reactive client-side validation response when an empty form is submitted. JavaScript intercepts the submission, checks input boundaries, and renders red border highlights alongside explanatory warning messages.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The main content area displays a registration form with a red border and a red background. It shows a single input field labeled "Email" on the left and "Address" on the right. The field contains the text "rahul.verma@" and has a red border. Below the field, a red error message reads: "Please include a valid domain such as '@gmail.com' or '@petcare.in'".

**Figure 1.3 Email Syntax Validation via Regular Expression Match**

This screenshot demonstrates the email validation mechanism. When a user enters 'rahul.verma@' without a domain suffix, JavaScript flags the string as non-compliant with standard format rules and immediately prompts for domain completion.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The main content area displays a registration form with a red border and a red background. It shows a single input field labeled "Phone" on the left, "Number" in the middle, and "(Mobile)" on the right. The field contains the text "98201ABCD" and has a red border. Below the field, a red error message reads: "Phone number must be exactly 10 digits without alphabetical characters."

**Figure 1.4 Phone Number Numeric and Length Constraint Enforcement**

This figure illustrates telephone validation enforcing an exact 10-digit numerical standard. Any alphabet character entry or incomplete digit sequence triggers an alert banner indicating invalid mobile format.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The page contains a registration form with two password fields. The first field, labeled "Password", has a green border and contains seven dots. The second field, labeled "Confirm", has a red border and also contains seven dots. To the right of the "Confirm" field, the word "Password" is displayed. Below the "Confirm" field, a red error message reads: "Passwords do not match. Please verify your entry."

**Figure 1.5 Password Mismatch Detection Between Password and Confirmation**

This screenshot displays the password parity verification logic. The confirmation field identifies that 'secret000' does not match the master password 'secret123', blocking premature form transmission.

A screenshot of a web browser window with the address bar showing `http://localhost:5000/register`. The page contains a role selection dropdown menu. The dropdown is labeled "Select" on the left and "Role" on the right. The selected option is "Pet Owner (Register personal pets, book visits)". Below the dropdown, a blue link reads: "Pet Owners gain access to patient history, passport creation, and booking."

**Figure 1.6 Role Selection Dropdown Specifying Pet Owner vs. Veterinarian**

This figure captures the dynamic account role selection interface, allowing the system to categorize registered actors into Pet Owners or Clinic Veterinarians with tailored downstream dashboard permissions.

http://localhost:5000/register

All fields conform to client validation rules

Full Name  
Tanuj Sharma

Email Address  
tanuj.sharma@petcare.org

Contact  
9876543210

Create Verified Account

**Figure 1.7 Validated Registration Form Ready for Database Ingestion**

This screenshot displays the completely validated form in an all-green compliant state with valid name, email, phone, and matching credentials, enabling the registration action button.

http://localhost:5000/register [Mobile View]

Mobile PetCare Register

Full Name

Email

Phone

Submit Mobile Form

**Figure 1.8 Responsive Mobile Viewport Rendering (Screen Simulation)**

This screenshot confirms the mobile responsiveness of the registration screen inspected via mobile emulation. The 12-column grid collapses fluidly, retaining full touch-target accessibility.

## **Conclusion:**

Experiment No. 1 successfully demonstrated the architecture, construction, and behavioral validation of a modern, responsive user registration portal using Bootstrap 5 and JavaScript ES6. The implementation of Bootstrap's twelve-column responsive grid system guaranteed complete visual harmony across varied device viewports, eliminating layout deformation.

Furthermore, decoupling data validation into modular client-side JavaScript functions significantly improved system responsiveness by intercepting non-compliant inputs before invoking server resources. The systematic application of regular expression patterns for email compliance, strict ten-

digit numeric checks for telephone records, and comparative password confirmation assertions ensured strong data integrity.

This experiment established the essential front-of-house onboarding foundation for the Pet Care Management System, validating how responsive design principles and client-side error handling converge to provide a professional, user-centric web experience.